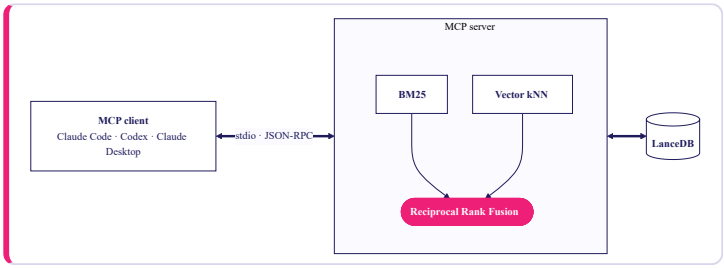
MCP-CUSTOMER-SUPPORT-TICKETS

MCP server for a 62k-ticket support corpus.

AUTHOR Tal Charnes **STACK** Python · FastMCP · Postgres + pgvector **DATASET** Tobi-Bueck/customer-support-tickets · ~62k · EN+DE
TRANSPORT stdio · local



One Python process backed by Postgres. Postgres `ts_rank_cd` (BM25-ish) and pgvector kNN (HNSW on cosine) run in parallel; **Reciprocal Rank Fusion** merges by rank position and returns a default top 10 (caller-tunable, hard cap 50) with stable `ticket://{id}` citations and an opaque cursor for resumable sweeps. Rows, a `tsvector` GIN index, an HNSW vector index, and 384-dim embeddings all live in one Postgres table.

01 Design

Three layers in one Python process. **Ingest** pulls the Hugging Face dataset, mints a 32-hex UUIDv7 per row as the primary `id`, and stamps the deterministic `sha1(revision‖row_index)[:12]` into an `original_system_id` column for cross-reference; then embeds rows with a small multilingual sentence-embedding model loaded on-device in a background warm-up thread (stdio handshake never blocks). **Store** is a single Postgres table holding rows, a `tsvector` GENERATED column with a GIN index, and a `vector(384)` column with an HNSW cosine index — pgvector and Postgres FTS in one place. **Serve** exposes nine tools — read-only ones (`search_tickets` with cursor pagination, `get_ticket`, `get_tickets` batch, `search_and_fetch` composite, `aggregate_tickets`, `server_info`) carry `readOnlyHint`; write tools (`create_ticket` with UUIDv7 ids and sha256 idempotency cache, `update_ticket`, `delete_ticket` with `destructiveHint`) carry a cross-tool replay warning. Two resources (`ticket://{id}`, `schema://tickets`) and one prompt — `draft_reply`, which assembles the target ticket plus up to five grounding neighbours (multi-language injection screen, body cap, markdown neutralization, target-language matching, prompt-structure invariants, weak-grounding hedge); the *client's* LLM writes the reply.

02 Why this solution

Hybrid beats either retriever alone — Postgres FTS (`ts_rank_cd`) catches literal tokens (error codes, SKUs); pgvector catches paraphrase; RRF fuses by rank position, so there are no weights to tune. Postgres co-locates rows, the `tsvector` GIN index, and HNSW vector index in one store the team already knows how to run, back up, and observe. Aggregates are plain SQL `GROUP BY`. **Avoided:** cross-encoder rerank (marginal once an LLM re-reads snippets), HyDE / query rewriting, chunking (tickets are already short), remote HTTP + OAuth, and any server-side LLM call — no API-key dependency, no hallucinated tickets.

03 In production

Live ticket source with incremental delta-ingest. Streamable HTTP transport with OAuth 2.1 and tenant-partitioned vector indices. A cross-encoder reranker on by default once retrieval feeds real workflows; a larger multilingual embedding model for richer coverage. PII redaction at ingest. An eval harness of 50–200 golden queries tracking precision@5 per deploy. Traces, p95 dashboards, per-tenant rate limits; signed images, CVE gating, and pinned model artifacts. **Next step beyond the current single-Postgres setup:** add **ClickHouse** alongside Postgres for analytical flows in prod — Postgres stays as OLTP for ticket CRUD and hybrid retrieval, while heavy rollups, time-series, and BI-style aggregates move to ClickHouse where they belong.

04 Risks

Hallucinated tickets. Read endpoints return dataset bytes verbatim; the server never calls an LLM. **Prompt injection via ticket bodies.** Content wrapped in `<ticket>` tags, a trust-boundary notice precedes every prompt, a multilingual heuristic screens both target and grounding before drafting, markdown is defanged to break clickable-link tricks, and an INTERNAL_ERROR envelope keeps raw tracebacks off the wire. **Dataset drift.** HF revision pinned into every `ticket://` URI and the embedding cache key; `is_valid` samples vectors for NaN/Inf before serving; re-index triggers on revision change. **Confident answers from weak matches.** `search_tickets` returns body snippets so the LLM judges from the text, not the rank; `draft_reply` refuses to scaffold unless a prior ticket clears cosine ≥ 0.70 with a non-empty answer; unsupported filters fail loud (no timestamp column).

05 Business impact

The same code, pointed at your own support corpus, lets reps and CSMs ask plain-English questions inside Claude or Cursor — *"how did we resolve the last few issues about X?"* — and get an answer linked back to the actual tickets. Institutional knowledge stops living only in individual reps' heads. **Swap the corpus and the pipeline doesn't change:** account-review notes, incident write-ups, internal runbooks — same retrieval, same citation contract. **Because the interface is MCP, one server is reachable from every AI surface** — Claude Code for engineers, Cursor and Codex for product, any internal agent — with no per-tool integration work. Build the retrieval surface once; every AI client gets it.

MCP-CUSTOMER-SUPPORT-TICKETS

Tal Charnes · 2026